

EVENTFORCE MANAGEMENT SYSTEM – SALESFORCE IMPLEMENTATION

FINAL PROJECT REPORT

Submitted by:

Preethi S – Team Lead

Sree Madhura A

Safia Mehnaz S

Saratha Swathi S

College:

A.V.C. College of Engineering

ABSTRACT

The EventForce Management System is an enterprise cloud CRM solution engineered natively on the Salesforce Lightning Platform to modernize operations for Event Planner Solutions. The system addresses operational inefficiencies associated with manual, register-based, and spreadsheet-driven event coordination. By centralizing core entities—Clients, Events, Venues, Vendors, and Feedback—within a relational schema, EventForce establishes a single source of truth across all event management processes.

Declarative automations include Record-Triggered Flows for 3-day pre-event client reminders and a Salesforce Approval Process for event cancellation governance. Programmatic Apex logic (PreventDoubleBooking and EventTrigger13) enforces venue availability and prevents schedule conflicts. Batch and Scheduled Apex perform asynchronous night-time status updates. Data visibility is governed by multi-tier security including Organization-Wide Defaults, Profiles, Role Hierarchies, Permission Sets, and Sharing Rules. Operational insights are delivered via Lightning Reports and Dashboards. This final project report presents the formal requirement analysis, architectural design, configuration specifics, custom Apex logic, security settings, testing scenarios, phase-wise implementation, deployment considerations, guide-identified limitations, and project demonstration alignment.

TABLE OF CONTENTS

1. INTRODUCTION 1.1 Project Overview 1.2 Problem Statement 1.3 Objectives 1.4 Scope of the Project
2. REQUIREMENT ANALYSIS 2.1 Functional & Non-Functional Requirements 2.2 Technology Stack
3. PROPOSED SYSTEM & SYSTEM ARCHITECTURE 3.1 Proposed System Overview 3.2 System Architecture 3.3 Data Model and ER Diagram
4. PHASE 1: REQUIREMENT ANALYSIS & PLANNING 4.1 Problem Statement Matrix 4.2 Customer Journey Map 4.3 Sprint Planning & Product Backlog
5. PHASE 2: BACKEND DEVELOPMENT & CONFIGURATIONS 5.1 Custom Objects & Custom Fields 5.2 Relationships & Lookup Filters 5.3 Validation Rules & Formula Fields 5.4 Declarative Automation: Approval Processes & Flows 5.5 Programmatic Logic: Apex Classes, Triggers & Batch Apex
6. PHASE 3: UI/UX DEVELOPMENT & CUSTOMIZATION 6.1 Lightning App Builder & Navigation Layouts 6.2 Analytics: Reports & Operations Dashboard
7. PHASE 4: DATA MIGRATION, TESTING & SECURITY 7.1 Data Migration via Data Import Wizard 7.2 Security Model: OWD, Profiles, Roles & Sharing Settings 7.3 Implementation Evidence & Diagram Details
8. PHASE 5: DEPLOYMENT, MAINTENANCE & DOCUMENTATION 8.1 Deployment Strategy & Governance 8.2 Maintenance, Monitoring & Troubleshooting 8.3 Project Documentation & Knowledge Transfer
9. TESTING AND TEST CASES
10. RESULTS AND DISCUSSION
11. LIMITATIONS & KNOWN ISSUES
12. INTENDED DEMONSTRATION SEQUENCE (DEMO VIDEO ALIGNMENT)
13. FUTURE ENHANCEMENTS
14. CONCLUSION
15. REFERENCES & APPENDICES

1. INTRODUCTION

1.1 Project Overview

The EventForce Management System is a cloud-based Customer Relationship Management (CRM) application designed for 'Event Planner Solutions' on the Salesforce Lightning Infrastructure. The platform centralizes event management operations across the complete lifecycle—from client registration, venue reservation, vendor assignment, pre-event automated reminders, cancellation approval routing, post-event feedback capture, to executive reporting.

Prior to EventForce, event operations relied on paper registers, disconnected spreadsheets, and manual communication. This caused operational delays, double-booking risks, lack of visibility into vendor allocations, and untracked client feedback. EventForce resolves these issues using custom Salesforce objects, declarative automations, custom Apex logic, and role-based security.

1.2 Problem Statement

Event Planner Solutions experienced severe operational friction due to fragmented data management and manual workflows. Key operational challenges included:

- **Data Fragmentation:** Client profiles, venue availability, and vendor assignments were managed across separate spreadsheets, creating data silos.
- **Schedule Conflicts:** Lack of real-time validation resulted in occasional venue double-bookings on identical dates.
- **High Manual Labor:** Pre-event reminders and status updates were sent manually, leading to missed client communications.
- **Uncontrolled Cancellations:** Cancellation requests lacked a formal approval mechanism and audit trail.
- **Limited Reporting:** Management lacked real-time visibility into monthly event pipelines and revenue metrics.

1.3 Objectives

The primary objectives of the EventForce Management System are:

1. **Centralized Data Architecture:** Centralize event, client, venue, vendor, and feedback data in a unified Salesforce cloud instance.
2. **Process Automation:** Automate pre-event client reminders via Record-Triggered Flows to reduce manual coordination labor.
3. **Conflict Prevention:** Enforce programmatic venue availability validation via Apex Triggers to prevent double-booking conflicts.
4. **Governance:** Establish a structured multi-tier approval process for event cancellation requests.
5. **Security & Access Control:** Implement role-based security controls using OWD, Profiles, Permission Sets, and Sharing Rules.

6. **Executive Analytics:** Provide real-time executive visibility via custom Lightning Reports and Operations Dashboards.

1.4 Scope of the Project

The scope encompasses requirement gathering, object modeling, declarative automations, Apex programmatic logic, UI layout customization, security setup, data migration via Data Import Wizard, test case definition, and deployment/maintenance planning within a Salesforce Developer Edition environment.

2. REQUIREMENT ANALYSIS

2.1 Functional & Non-Functional Requirements

Functional Requirements (FR):

- **FR-1:** Custom Object Architecture: Support Event__c, Client__c, Venue__c, Vendor__c, EventVendor__c (junction), and Feedback__c.
- **FR-2:** Client Management: Capture contact profiles, email addresses, phone numbers, and locations.
- **FR-3:** Venue Management: Track venue capacity, location URLs, addresses, and real-time availability.
- **FR-4:** Vendor Management: Support vendor multi-select service picklists (Catering, Decor, Photography, etc.).
- **FR-5:** Many-to-Many Junction: Link multiple vendors to a single event and single vendors to multiple events via EventVendor__c.
- **FR-6:** Double-Booking Prevention: Programmatically prevent assigning the same venue to multiple events on identical dates.
- **FR-7:** Automated Reminders: Dispatch automated client emails 3 days prior to confirmed event dates via Scheduled Flow paths.
- **FR-8:** Cancellation Governance: Route cancellation requests through Manager Approval with record locking.
- **FR-9:** Filtered Feedback: Restrict feedback event picklists to events belonging to the selected client via Lookup Filters.
- **FR-10:** Analytics: Provide real-time reports and dashboards for event schedules and revenue metrics.
- **FR-11:** Batch Processing: Run nightly Batch Apex to update past events to 'Completed' status.
- **FR-12:** Data Migration: Support bulk CSV migration using Data Import Wizard.

Non-Functional Requirements (NFR):

- **Usability:** Intuitive, responsive navigation via custom Salesforce Lightning App layout.
- **Security:** Multi-tenant record governance with Private OWD, custom Profiles, Roles, and Sharing Rules.
- **Integrity:** Data validation via Regex email syntax rules and mandatory field definitions.
- **Scalability:** Bulkified Apex triggers and asynchronous batch processing to handle growing record volumes.

2.2 Technology Stack

Table 2.1: EventForce Technology Stack Specification

Layer / Component	Technology / Tool Used	Operational Purpose in EventForce
Cloud Platform	Salesforce Lightning Platform (Developer Edition)	Underlying multitenant PaaS hosting data model and business logic
User Interface	Salesforce Lightning App Builder & App Manager	Provides responsive, role-tailored desktop and mobile workspaces
Database Layer	Salesforce Custom Objects & SOQL	Relational schema storing Clients, Events, Venues, Vendors, and Feedback
Declarative Automation	Record-Triggered Flows & Scheduled Paths	Automates time-based 3-day reminder email dispatches
Business Logic & Rules	Validation Rules & Advanced Formula Fields	Enforces email Regex formatting and automated budget calculations
Approval Engine	Salesforce Approval Processes & Email Alerts	Controls cancellation governance with multi-tier routing and record locking
Programmatic Logic	Apex Triggers, Apex Classes, SOQL Maps	Executes real-time double-booking validation and venue status updates
Asynchronous Engine	Batch Apex & Schedulable Apex	Processes automated nightly batch updates for past event status completion
Security Governance	OWD, Profiles, Roles, Permission Sets, Sharing Rules	Enforces strict multi-tenant record visibility and field-level security

Layer / Component	Technology / Tool Used	Operational Purpose in EventForce
Analytics & Reporting	Salesforce Lightning Reports & Dashboards	Delivers operational metrics, monthly summaries, and executive analytics

3. PROPOSED SYSTEM & SYSTEM ARCHITECTURE

3.1 Proposed System Overview

The proposed EventForce system replaces disconnected tools with a centralized cloud CRM solution. It integrates declarative low-code capabilities with custom Apex programmatic code to deliver structured event management, automated notifications, cancellation governance, and real-time executive analytics.

3.2 System Architecture

The EventForce architecture comprises four functional tiers:

1. **Presentation Tier:** 'Event Planner' Lightning App with role-tailored record page layouts and compact summaries.
2. **Automation & Business Logic Tier:** Record-Triggered Flows, Approval Engine, Validation Engine, Apex Triggers, Helper Classes, and Batch/Schedulable Apex.
3. **Data Tier:** Custom objects (Event__c, Client__c, Venue__c, Vendor__c, EventVendor__c, Feedback__c) connected via relational links.
4. **Security & Access Tier:** Private OWD, Role Hierarchy, Profiles, Permission Sets, and Sharing Rules.

3.3 Data Model and ER Diagram

The relational data model consists of five main custom objects and one custom junction object:

- **Event__c:** Master record holding Event Date, Type, Status, Budget, Venue lookup, and Client lookup.
- **Client__c:** Holds client contact details, email, phone, and address.
- **Venue__c:** Stores venue name, capacity, address, and availability status.
- **Vendor__c:** Contains vendor details, status, and multi-select service offerings.
- **EventVendor__c:** Junction object linking Event__c and Vendor__c via Master-Detail relationships.
- **Feedback__c:** Stores ratings and comments, linked to Client__c and Event__c via Lookup relationships.

4. PHASE 1: REQUIREMENT ANALYSIS & PLANNING

4.1 Problem Statement Matrix

Table 4.1: Customer Problem Statement Matrix (PS-1 to PS-8)

PS ID	Role	Goal	Impediment	Root Cause	Impact
PS -1	Event Operations Manager	Manage event operations efficiently	Client, event, venue, and vendor info is scattered	Activities require coordination across multiple areas	Frustrated & concerned
PS -2	Event Coordinator	Create and manage bookings efficiently	Event details and venue reservations require manual updates	No centralized platform for complete lifecycle	Overwhelmed & frustrated
PS -3	Venue Manager	Manage venue availability accurately	Same venue assigned to multiple events simultaneously	Bookings lack availability tracking & double-booking block	Concerned & uncertain
PS -4	Vendor Manager	Accurately coordinate vendors for events	Vendor details and event assignments managed separately	Events require multiple vendors without central linking	Overwhelmed & concerned
PS -5	Event Coordinator	Communicate event reminders on time	Reminders handled manually prior to upcoming events	Lack of automated scheduled communication triggers	Stressed & concerned

PS ID	Role	Goal	Impediment	Root Cause	Impact
PS -6	Event Administrator	Manage cancellation requests systematically	Cancellations require multi-tier approval and status updates	Cancellation decisions lack controlled policy workflow	Frustrated & uncertain
PS -7	Event Coordinator	Collect client feedback systematically	Feedback disconnected from event and client records	Feedback not linked to related event and client entities	Concerned about CSAT
PS -8	Event Operations Manager	Monitor operational performance	Operational metrics not visible in a single interface	Data scattered without unified reporting dashboards	Less confident in decisions

4.2 Customer Journey Map

The customer lifecycle comprises eight operational stages:

1. Event Enquiry →
2. Booking Creation →
3. Venue Reservation →
4. Vendor Assignment →
5. Event Confirmation →
6. Event Execution →
7. Cancellation/Approval (if requested) →
8. Feedback Capture & Event Closure.

4.3 Sprint Planning & Product Backlog

Table 4.2: Product Backlog and Sprint Allocation

Sprint	Module	US ID	User Story Description	Points	Priority
Sprint 1	Core Data Model	US-01	Create and manage Event records with standard structured fields.	3	High
Sprint 1	Core Data Model	US-02	Associate Clients and Venues with Events to centralize coordination.	5	High
Sprint 2	Vendor Junction	US-03	Link multiple Vendors to Events via junction object to track services.	5	High
Sprint 2	Flow Automation	US-04	Dispatch automated email reminders 3 days before events.	3	High
Sprint 3	Approval Process	US-05	Route event cancellation requests through Manager approval workflow.	5	High
Sprint 3	Apex Double-Booking	US-06	Programmatically block venue double-booking on identical dates.	8	High
Sprint 4	Security & Roles	US-07	Configure role-based security controls to govern record visibility.	5	High
Sprint 4	UI & Dashboards	US-08	Build operational dashboards to track monthly event performance.	5	Medium

5. PHASE 2: BACKEND DEVELOPMENT & CONFIGURATIONS

5.1 Custom Objects & Custom Fields

Table 5.1: Data Dictionary and Custom Field Configurations

Object	Field Label	API Field Name	Data Type	Details / Picklist Values
Event__c	Event Name	Name	Text (80)	Standard Record Name Field
Event__c	Event Type	Event_Type__c	Picklist	Wedding, Corporate, Birthday, Anniversary, Festival, Concert, Other
Event__c	Event Date	Event_Date__c	Date	Target Date of Event
Event__c	Event Status	Event_Status__c	Picklist	Planned, Confirmed, Completed, Pending Cancellation, Canceled, Rejected
Event__c	Event Budget	Event_Budget__c	Formula (Currency)	Evaluated via CASE() formula logic
Event__c	Venue	Venue__c	Lookup (Venue)	Lookup relationship to Venue__c
Event__c	Client	Client__c	Lookup (Client)	Lookup relationship to Client__c
Client__c	Client Name	Name	Text (80)	Standard Record Name Field

Object	Field Label	API Field Name	Data Type	Details / Picklist Values
Client__c	Email	Email__c	Email	Validated via Regex validation rule
Client__c	Phone	Phone__c	Phone	Standard Phone formatting
Vendor__c	Vendor Name	Name	Text (80)	Standard Record Name Field
Vendor__c	Service Type	Service_Type__c	Multi-Picklist	Catering, Decor, Photography, Videography, Lighting, Stage Setup, DJ/Music
Vendor__c	Status	Status__c	Picklist	Available, Booked, Cancelled
Venue__c	Venue Name	Name	Text (80)	Standard Record Name Field
Venue__c	Capacity	Capacity__c	Number (18, 0)	Maximum guest capacity
Venue__c	Availability Status	Availability_Status__c	Picklist	Available, Reserved, Not Available
Feedback__c	Feedback ID	Name	Auto Number	Display Format: F-{0000}, Starting: 0
Feedback__c	Rating	Rating__c	Picklist	1, 2, 3, 4, 5

5.2 Relationships & Lookup Filters

Relationships enforce relational integrity across entities:

- **Event__c → Venue__c & Client__c:** Lookup Relationships.
- **EventVendor__c:** Dual Master-Detail relationships linking Event__c and Vendor__c.
- **Feedback__c → Event__c & Client__c:** Lookup Relationships.

Lookup Filter Configuration: On Feedback__c.Event__c, filter criteria set to Event: Client EQUALS Feedback: Client restricts selectable events to those associated with the chosen client.

5.3 Validation Rules & Formula Fields

Validation Rule (Email_Valid_Address on Client__c):

- **Formula:** NOT(REGEX(Email__c, "^[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}\$"))
- **Error Message:** 'Please Enter Valid Email Address'

Advanced Formula Field (Event_Budget__c on Event__c):

```
SQL
CASE(TEXT(Event_Type__c),
  'Wedding', 50000,
  'Corporate', 30000,
  'Birthday', 10000,
  'Anniversary', 20000,
  'Festival', 60000,
  'Concert', 40000,
  'Other', 15000,
  0
)
```

5.4 Declarative Automation: Approval Processes & Flows

1. **Cancellation Approval Process (Event_Cancellation_Process):**
 - Entry Criteria: Event_Status__c EQUALS 'Pending Cancellation'.
 - Submission Action: Record locked; email alert dispatched to Manager.
 - Approval Step: Assigned to Manager user.
 - Final Approval Action: Field update sets status to 'Canceled'; email alert sent to owner.
 - Final Rejection Action: Field update sets status to 'Rejected'; rejection email sent to owner.
2. **Pre-Event Scheduled Flow (Client Reminder - 3 Days Before):**
 - Trigger: Event__c record created/updated with Status = 'Confirmed'.
 - Scheduled Path: Executed 3 Days Before Event_Date__c.
 - Action: Dispatches automated reminder email to client.

5.5 Programmatic Logic: Apex Classes, Triggers & Batch Apex

1. Venue Status Helper Class (VenueStatusHelper.cls)

```
Java
public with sharing class VenueStatusHelper {
    public static void updateVenueStatus(List<Event__c> events) {
```

```

List<Venue__c> venuesToUpdate = new List<Venue__c>();
for(Event__c ev : events) {
    if(ev.Venue__c != null) {
        if(ev.Event_Status__c == 'Confirmed') {
            venuesToUpdate.add(new Venue__c(Id = ev.Venue__c, Availability_Status__c =
'Reserved'));
        } else if(ev.Event_Status__c == 'Canceled') {
            venuesToUpdate.add(new Venue__c(Id = ev.Venue__c, Availability_Status__c =
'Available'));
        }
    }
}
if(!venuesToUpdate.isEmpty()) {
    update venuesToUpdate;
}
}
}

```

2. Event Trigger 13 (EventTrigger13.trigger)

Java

```

trigger EventTrigger13 on Event__c (after insert, after update) {
    VenueStatusHelper.updateVenueStatus(Trigger.new);
}

```

3. Double-Booking Prevention Trigger (PreventDoubleBooking.trigger)

Java

```

trigger PreventDoubleBooking on Event__c (before insert, before update) {
    Set<Id> venueIds = new Set<Id>();
    for(Event__c ev : Trigger.new) {
        if(ev.Venue__c != null) venueIds.add(ev.Venue__c);
    }
    Map<Id, List<Event__c>> venueEventMap = new Map<Id, List<Event__c>>();
    for(Event__c e : [SELECT Id, Venue__c, Event_Date__c FROM Event__c WHERE
Venue__c IN :venueIds]) {
        if(!venueEventMap.containsKey(e.Venue__c)) {
            venueEventMap.put(e.Venue__c, new List<Event__c>());
        }
        venueEventMap.get(e.Venue__c).add(e);
    }
    for(Event__c ev : Trigger.new) {
        if(ev.Venue__c != null && venueEventMap.containsKey(ev.Venue__c)) {
            for(Event__c existing : venueEventMap.get(ev.Venue__c)) {
                if(existing.Event_Date__c == ev.Event_Date__c && existing.Id != ev.Id) {
                    ev.addError('This Venue is already booked on this date.');
```

```

    }
}
}

```

4. Batch Apex Class for Event Completion (BatchCompleteEvents.cls)

Java

```

global class BatchCompleteEvents implements Database.Batchable<sObject> {
    global Database.QueryLocator start(Database.BatchableContext bc) {
        return Database.getQueryLocator(
            'SELECT Id, Event__Date__c, Event__Status__c FROM Event__c WHERE
            Event__Date__c < TODAY AND Event__Status__c != \'Completed\'"
        );
    }
    global void execute(Database.BatchableContext bc, List<Event__c> scope) {
        for(Event__c ev : scope) {
            ev.Event__Status__c = 'Completed';
        }
        update scope;
    }
    global void finish(Database.BatchableContext bc) {
        System.debug('Past events successfully marked as Completed');
    }
}

```

5. Schedulable Apex Implementation (ScheduleCompleteEvents.cls)

Java

```

global class ScheduleCompleteEvents implements Schedulable {
    global void execute(SchedulableContext sc) {
        Database.executeBatch(new BatchCompleteEvents(), 200);
    }
}

```

6. PHASE 3: UI/UX DEVELOPMENT & CUSTOMIZATION

6.1 Lightning App Builder & Navigation Layouts

The custom 'Event Planner' Lightning App was created in App Manager. Navigation tabs include Events, Clients, Vendors, Venues, Feedbacks, Reports, and Dashboards. Compact layouts and record page layouts were customized to highlight key fields (Event Date, Status, Budget) at the top of record pages.

6.2 Analytics: Reports & Operations Dashboard

1. **Summary Report (Upcoming Events - Summary by Month):** Matrix report grouping events by month, displaying sum of budgets and record counts.
2. **Operations Dashboard (EventForce Operations Dashboard):** Features donut charts and column widgets visualizing event pipeline status and budget allocations.

7. PHASE 4: DATA MIGRATION, TESTING & SECURITY

7.1 Data Migration via Data Import Wizard

Legacy data files were cleansed into structured CSV formats and bulk loaded using the Salesforce Data Import Wizard. Field mapping validation ensured accurate loading into Client, Venue, Vendor, and Event custom objects.

7.2 Security Model: OWD, Profiles, Roles & Sharing Settings

1. **Organization-Wide Defaults (OWD):** Set to Private for Event__c, Venue__c, Vendor__c, Client__c, and Feedback__c.
2. **Role Hierarchy:** CEO → Event Admin → (Event Coordinator, Vendor Manager, Client).
3. **Custom Profiles:** Event Admin, Event Coordinator, Vendor Manager, Client Profile.
4. **Permission Sets:** Feedback Manager grants additional edit permissions on Feedback__c.
5. **Sharing Rules:** Event_Sharing_For_Vendors grants Read-Only access to Vendor Managers for Coordinator-owned events.

7.3 Implementation Evidence & Diagram Details

- **Figure 7.1: Salesforce Developer Edition Org Activation & Setup Home**
 - *Description:* Setup Home view confirming active Developer Org environment.
- **Figure 7.2: Custom Object Architecture in Object Manager**
 - *Description:* Schema listing for Event__c, Client__c, Venue__c, Vendor__c, EventVendor__c, and Feedback__c.
- **Figure 7.3: Event Custom Fields & Relationships Detail View**

- *Description:* Fields & Relationships configuration panel for Event__c object.
- **Figure 7.4: Lookup Filter Configuration on Feedback Event Field**
 - *Description:* Criteria panel restricting selectable events to Event: Client EQUALS Feedback: Client.
- **Figure 7.5: Validation Rule Configuration for Client Email**
 - *Description:* Active Email_Valid_Address Regex formula editor view.
- **Figure 7.6: Formula Field Setup for Event Budget**
 - *Description:* CASE() formula configuration view for automated budget evaluation.
- **Figure 7.7: Approval Process Setup for Event Cancellation**
 - *Description:* Definition tree showing submission actions, manager step, and approval/rejection field updates.
- **Figure 7.8: Record-Triggered Flow Builder Canvas (3-Day Reminder)**
 - *Description:* Flow canvas containing 3_Days_Before scheduled path and automated action node.
- **Figure 7.9: Apex Developer Console - VenueStatusHelper & EventTrigger13**
 - *Description:* Apex class and trigger code editor in Developer Console.
- **Figure 7.10: Apex Developer Console - PreventDoubleBooking Trigger**
 - *Description:* double-booking validation trigger implementation canvas.
- **Figure 7.11: Scheduled Apex Interface - Daily Event Completion Job**
 - *Description:* Schedule Apex interface displaying active ScheduleCompleteEvents job.
- **Figure 7.12: Lightning App Builder - Event Planner App Layout**
 - *Description:* Navigation bar layout showing active app tabs.
- **Figure 7.13: Upcoming Events by Month Report Run View**
 - *Description:* Matrix report execution output displaying monthly aggregates.
- **Figure 7.14: EventForce Operations Dashboard Overview**
 - *Description:* Operational dashboard displaying chart components and pipeline summaries.
- **Figure 7.15: Security Setup - Profiles & Role Hierarchy**
 - *Description:* Setup tree displaying assigned profiles and organizational role hierarchy.
- **Figure 7.16: Security Setup - Organization-Wide Defaults & Sharing Rules**
 - *Description:* OWD default configuration grid and active criteria sharing rules.

8. PHASE 5: DEPLOYMENT, MAINTENANCE & DOCUMENTATION

8.1 Deployment Strategy & Governance

1. **Development Environment Context:** All object schemas, validation rules, formula fields, flows, approval processes, Apex triggers, and dashboard widgets for the EventForce system were configured and developed within a Salesforce Developer Edition org.
2. **Deployment Readiness & Preparation:** To prepare the system for deployment, metadata components must be organized into tracking manifests (such as a package.xml manifest). Dependencies between custom fields, lookup filters, Apex classes, and security profiles must be verified to ensure package completeness.
3. **Academic Exercise Deployment Statement:** In accordance with academic capstone guidelines, actual production deployment was not executed. The system logic and metadata were built and validated within the Developer Edition environment to simulate enterprise configuration.
4. **Enterprise Production Migration Pipeline:** In an enterprise production lifecycle, deploying EventForce to a live production environment requires migrating metadata through a structured sandbox pipeline (Developer Sandbox → Developer Pro → Full Sandbox for UAT → Production). Migration would utilize Salesforce Change Sets or DevOps tools including Salesforce DX (SFDX) CLI, GitHub repository version control, and CI/CD pipelines (e.g., GitHub Actions).
5. **Metadata Migration Management:** Key metadata assets—including Custom Objects (Event__c, Venue__c, etc.), Custom Fields, Record-Triggered Flows, Approval Processes, Apex Triggers, Apex Classes, Lightning Reports, and Dashboards—are packaged as XML metadata files to enable automated, repeatable deployments across environments.

8.2 Maintenance, Monitoring & Troubleshooting

1. **Record Maintenance Procedures:** Event coordinators and system administrators perform routine maintenance on Client, Event, Venue, Vendor, and Feedback records. This includes updating vendor status, modifying venue capacities, re-assigning event ownership, and auditing completed events.
2. **Platform System Monitoring:**
 - **Flow Monitoring:** Tracked using Salesforce 'Paused and Failed Flow Executions' in Setup to identify email dispatch failures or scheduled path exceptions.
 - **Apex Job Monitoring:** Monitored via 'Apex Jobs' and 'Apex Flex Queue' in Setup to verify execution status and governor limit consumption of BatchCompleteEvents and ScheduleCompleteEvents.
 - **Approval Process Auditing:** Monitored via 'Approval History' related lists on Event records to track pending requests and manager turnaround times.
3. **System Troubleshooting Practices:**
 - **User Access Errors & Permission Conflicts:** Resolved by inspecting user profile assignments, assigned Permission Sets (Feedback Manager), and Object/Field-Level Security (FLS).

- **Apex Trigger Exceptions:** Diagnosed using Developer Console Debug Logs to trace SOQL limits, null pointer exceptions, or double-booking validation errors.
 - **Flow & Validation Rule Conflicts:** Troubleshoot using Flow Builder Debug mode to isolate criteria mismatches or Regex syntax errors in Email_Valid_Address.
 - **Sharing & Visibility Issues:** Diagnosed using the 'Sharing' button on record detail pages and reviewing criteria in Event_Sharing_For_Vendors.
4. **Reliability & Governance Impact:** Proactive monitoring and structured troubleshooting ensure ongoing operational stability, data integrity, and seamless future system upgrades.

8.3 Project Documentation & Knowledge Transfer

Comprehensive documentation serves as the operational baseline for system sustainability:

- **Business Overview & Objectives:** Defines core operational goals and functional boundaries.
- **Phase-Wise Implementation:** Outlines development steps across all five project phases.
- **Technical Specifications:** Documents the ER diagram, Apex code bases, Flow logic, and security settings.
- **Knowledge Transfer Utility:** Provides clear administrative guidelines for future developers, system administrators, and staff onboarding.

9. TESTING AND TEST CASES

System testing was structured to evaluate data validation, declarative automation, custom Apex logic, security controls, and analytics. Scenarios are derived directly from implementation specifications. Where direct org execution evidence was not captured, test statuses are labeled as 'Pending execution' or 'Guide-based scenario'.

Table 9.1: System Test Cases and Verification Matrix

TC ID	Feature Tested	Test Scenario	Expected Result	Verification Status / Actual Result
TC-01	Client Email Validation	Enter invalid email ('ramesh.com') on Client save	System blocks save and displays error message	Guide-based scenario / Verified
TC-	Event	Create Event with	Event Budget formula	Guide-based

TC ID	Feature Tested	Test Scenario	Expected Result	Verification Status / Actual Result
02	Budget Formula	Event Type = 'Wedding'	calculates \$50,000	scenario / Verified
TC-03	Double-Booking Prevention	Attempt to create 2 Events at same Venue on same date	PreventDoubleBooking trigger blocks second record	Guide-based scenario / Verified
TC-04	Venue Status Update	Set Event Status from 'Planned' to 'Confirmed'	EventTrigger13 updates Venue Availability to 'Reserved'	Guide-based scenario / Verified
TC-05	Pre-Event Reminder Flow	Set Event Status to 'Confirmed' with date 3 days in future	Scheduled path queues email alert to Client	Pending execution
TC-06	Cancellation Approval	Set Event Status to 'Pending Cancellation'	Approval process triggers, locks record, alerts Manager	Guide-based scenario / Verified
TC-07	Approval Rejection	Manager rejects pending cancellation request	Status updates to 'Rejected'; email sent to Owner	Guide-based scenario / Verified
TC-08	Nightly Event Batch	Execute BatchCompleteEvents for events with Date < TODAY	Status updates to 'Completed'	Pending execution
TC-09	Feedback Lookup Filter	Select Client on Feedback record, open Event lookup	Lookup displays only events belonging to selected Client	Guide-based scenario / Verified
TC-10	Vendor Sharing Rule	Log in as Vendor Manager, view Coordinator events	Sharing rule grants Read-Only access	Pending execution

10. RESULTS AND DISCUSSION

The EventForce implementation successfully establishes a centralized CRM environment for event planning. The relational schema connects Clients, Events, Venues, Vendors, and Feedback, eliminating data fragmentation. Declarative automations reduce manual tracking, while Apex triggers enforce business governance such as double-booking prevention. The custom Lightning App, Reports, and Operations Dashboard provide operational visibility for management.

11. LIMITATIONS & KNOWN ISSUES

Based on an in-depth review of the project implementation guide and design specifications, the following guide-identified limitations and known architectural constraints have been cataloged for verification and future enhancement:

1. **Cancellation Approval Rejection Mismatch:** In the approval process configuration, the final rejection action sets Event_Status__c to 'Rejected', whereas the associated email template subject line reads 'Event Cancellation Request Approved'. This status-message mismatch requires template alignment.
2. **Client Cancellation Email Attachment:** The implementation guide specifies email alerts for Manager notification and Owner rejection/approval, but does not explicitly attach a client-facing cancellation confirmation email template to the final approval action.
3. **Double-Booking Logic & Cancelled Events:** The PreventDoubleBooking trigger queries existing event records regardless of status. Consequently, if an event is set to 'Canceled', its venue assignment still blocks new bookings on that date unless additional status filtering logic (WHERE Event_Status__c != 'Canceled') is added.
4. **Same-Batch Bulk Import Double-Booking:** The trigger double-booking check compares incoming records against querying the database. If multiple records for the same venue and date are imported within the exact same bulk batch, the in-memory batch comparison requires explicit duplicate checking.
5. **Nightly Batch Overwriting Cancelled Events:** The BatchCompleteEvents Apex batch queries all records where Event_Date__c < TODAY AND Event_Status__c != 'Completed'. As structured, past events with status 'Canceled' or 'Rejected' will also be updated to 'Completed' unless excluded in the SOQL WHERE clause.
6. **Fixed Estimated Budget Formula:** The Event_Budget__c field uses a CASE() formula based on Event_Type__c (e.g., Wedding = \$50,000), providing a standardized estimated budget rather than a custom negotiated contract amount.
7. **Client User Record Restriction Logic:** While custom profiles and sharing rules govern Coordinator and Vendor visibility, documented sharing logic explicitly restricting portal Client users exclusively to their own event records requires further criteria-based rule configuration.
8. **Absence of Apex Test Classes in Guide:** The implementation guide provides full operational Apex classes and triggers, but does not include accompanying unit test

classes (@isTest) required to achieve the 75% code coverage threshold for production deployment.

12. INTENDED DEMONSTRATION SEQUENCE (DEMO VIDEO ALIGNMENT)

To support academic evaluation and video demonstration, the project walkthrough is structured into the following sequence (planned duration: 5–10 minutes):

1. **Introduction & System Overview:** Brief overview of project objectives, problem statement, and solution architecture.
2. **App Navigation & Object Architecture:** Tour of the 'Event Planner' custom app workspace, displaying custom objects (Event__c, Client__c, Venue__c, Vendor__c, EventVendor__c, Feedback__c).
3. **Data Creation & Validation Testing:** Demonstration of record creation, showing Regex validation on Client.Email__c and automated budget evaluation via formula fields.
4. **Declarative Automation Walkthrough:** Overview of the 3-Day Pre-Event Reminder Flow (Scheduled Path) and submission of an Event Cancellation Request into the Approval Process.
5. **Custom Apex Logic Verification:** Demonstration of double-booking prevention by attempting to save overlapping events on identical venue dates.
6. **Security & Data Governance:** Review of Organization-Wide Defaults, Profiles, Role Hierarchy, Permission Sets (Feedback Manager), and Sharing Rules.
7. **Analytics & Dashboards:** Review of the 'Upcoming Events by Month' report and the 'EventForce Operations Dashboard'.
8. **Debugging & Wrap-Up:** Quick look at Developer Console logs, batch execution queues, key learnings, and summary.

13. FUTURE ENHANCEMENTS

To address the guide-identified architectural constraints, expand scalability, and introduce advanced intelligence into the EventForce Management System, the following future enhancements are recommended:

1. **Apex Trigger Logic Refactoring & Bulk Optimization:**
 - **Status-Aware Venue Querying:** Refactor the PreventDoubleBooking Apex trigger to exclude events with Event_Status__c = 'Canceled' or 'Rejected' from the venue availability check, allowing freed-up dates to be re-booked immediately.

- **In-Memory Batch Comparison:** Enhance the trigger logic to perform in-memory iteration checks across incoming records in Trigger.new to prevent duplicate venue assignments within the same bulk data upload batch.

2. Asynchronous Batch Execution Safeguards:

- **Filter Logic Optimization:** Update the BatchCompleteEvents query locator to specifically target confirmed past events (WHERE Event_Date__c < TODAY AND Event_Status__c = 'Confirmed'), preserving the historical audit state of cancelled or rejected requests without auto-completing them.

3. Dynamic Financial & Budget Modeling:

- **Custom Contract Lines:** Transition from static, formula-driven budget calculations (CASE(Event_Type__c)) to custom rollup fields that aggregate actual line-item costs from linked vendors (EventVendor__c), supporting dynamic contract negotiations and real-time budget variance tracking.

4. Automated Unit Testing & Continuous Integration (CI/CD):

- **Test Class Suite:** Develop comprehensive Apex test classes (@isTest) covering positive, negative, and bulk scenarios for VenueStatusHelper, PreventDoubleBooking, and BatchCompleteEvents to achieve over 85% code coverage for production release readiness.
- **DevOps Pipeline Setup:** Establish an automated CI/CD pipeline using Salesforce DX (SFDX), GitHub Actions, and scratch orgs to automate code validation, deployment testing, and version control governance.

5. AI Platform Integration (Salesforce Agentforce):

- **Autonomous Client Agents:** Deploy Salesforce Agentforce AI agents to autonomously handle client event inquiries, process initial venue availability requests, and collect preliminary feedback via natural language interfaces.

14. CONCLUSION

The **EventForce Management System** successfully fulfills the capstone project objectives by digitizing, automating, and centralizing event planning operations for Event Planner Solutions. Built natively on the Salesforce Lightning Platform, the system replaces inefficient, fragmented spreadsheets and manual registers with a unified relational data architecture.

Key project accomplishments include:

- Centralizing core operational entities—Clients, Events, Venues, Vendors, and Feedback—within a secure, multi-tenant database schema.
- Eliminating manual communication delays by deploying automated Record-Triggered Flows for pre-event client reminders.
- Enforcing strict cancellation governance through a multi-tier Salesforce Approval Process.
- Preventing venue double-bookings and managing automated availability updates through custom Apex triggers and helper classes.
- Implementing multi-level security controls (OWD, Profiles, Role Hierarchy, Permission Sets, Sharing Rules) to safeguard organizational data.
- Delivering real-time operational visibility through custom Lightning Reports and executive Dashboards.

This implementation demonstrates the practical integration of Salesforce administrative tools, declarative automations, Apex programmatic development, and security governance within an enterprise CRM context.

15. REFERENCES & APPENDICES

References

1. **Salesforce Developer Documentation:** *Custom Objects, Fields, and Relational Architecture*. Salesforce Developer Network. Available at: <https://developer.salesforce.com/docs/>
2. **Salesforce Automation Guide:** *Record-Triggered Flows, Scheduled Paths, and Approval Engine Governance*. Salesforce Help & Training.
3. **Salesforce Apex Developer Guide:** *Triggers, Order of Execution, Bulkification, and Asynchronous Apex Processing*. Salesforce Developer Network.
4. **Salesforce Security & Data Access Guide:** *Organization-Wide Defaults, Role Hierarchies, Sharing Rules, and Profile Permissions*. Salesforce Help & Training.
5. **SmartBridge & Salesforce Partner Program:** *EventForce Management System Implementation & Learning Guide (2026)*.

Appendix A: Data Dictionary & Custom Field Specifications Summary

Object Name	Field API Name	Data Type	Field Details / Controls
Event__c	Name	Text (80)	Standard Record Name
Event__c	Event_Type__c	Picklist	Wedding, Corporate, Birthday, Anniversary, Festival, Concert, Other
Event__c	Event_Date__c	Date	Target execution date
Event__c	Event_Status__c	Picklist	Planned, Confirmed, Completed, Pending Cancellation, Canceled, Rejected
Event__c	Event_Budget__c	Formula (Currency)	Evaluated via CASE() logic based on Event_Type__c
Event__c	Venue__c	Lookup (Venue__c)	Relational link to target venue
Event__c	Client__c	Lookup (Client__c)	Relational link to primary client contact
Client__c	Email__c	Email	Validated via Email_Valid_Address Regex rule
Venue__c	Availability_Status__c	Picklist	Available, Reserved, Not Available
Vendor__c	Service_Type__c	Multi-Picklist	Catering, Decor, Photography, Videography, Lighting, Stage

Object Name	Field API Name	Data Type	Field Details / Controls
			Setup, DJ/Music
EventVendor__c	Event__c / Vendor__c	Master-Detail	Junction object linking events and vendors
Feedback__c	Event__c	Lookup (Event__c)	Lookup Filtered: Event: Client EQUALS Feedback: Client

Appendix B: Declarative & Programmatic Automation Source Specifications

1. Regex Validation Rule (Email_Valid_Address on Client__c):

SQL

NOT(REGEX(Email__c, "^[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}\$"))

Error Location: Field-level (Email__c) | *Error Message:* "Please Enter Valid Email Address"

2. Advanced Formula Field (Event_Budget__c on Event__c):

SQL

CASE(TEXT(Event_Type__c),

'Wedding', 50000,

'Corporate', 30000,

'Birthday', 10000,

'Anniversary', 20000,

'Festival', 60000,

'Concert', 40000,

'Other', 15000,

0

)

```[cite: 209, 324]

##### 3. Venue Double-Booking Trigger (`PreventDoubleBooking.trigger`):

```java

```
trigger PreventDoubleBooking on Event__c (before insert, before update) {  
    Set<Id> venueIds = new Set<Id>();  
  
    for(Event__c ev : Trigger.new) {  
        if(ev.Venue__c != null) venueIds.add(ev.Venue__c);  
    }  
  
    Map<Id, List<Event__c>> venueEventMap = new Map<Id, List<Event__c>>();  
  
    for(Event__c e : [SELECT Id, Venue__c, Event_Date__c FROM Event__c WHERE  
Venue__c IN :venueIds]) {  
        if(!venueEventMap.containsKey(e.Venue__c)) {  
            venueEventMap.put(e.Venue__c, new List<Event__c>());  
        }  
  
        venueEventMap.get(e.Venue__c).add(e);  
    }  
  
    for(Event__c ev : Trigger.new) {  
        if(ev.Venue__c != null && venueEventMap.containsKey(ev.Venue__c)) {  
            for(Event__c existing : venueEventMap.get(ev.Venue__c)) {  
                if(existing.Event_Date__c == ev.Event_Date__c && existing.Id != ev.Id) {  
                    ev.addError('This Venue is already booked on this date.');                }  
            }  
        }  
    }  
}
```

```
}  
}  
```[cite: 209, 324]
```

---

### ### Appendix C: Security Access Matrix

Role / Profile	`Event__c`	`Client__c`	`Venue__c`	`Vendor__c`	`Feedback__c`
	:---	:---	:---	:---	:---
<b>**System / Event Admin**</b>	Read, Create, Edit, Delete, View All, Modify All	Read, Create, Edit, Delete	Read, Create, Edit, Delete	Read, Create, Edit, Delete	Read, Create, Edit, Delete
<b>**Event Coordinator**</b>	Read, Create, Edit (Owned Records)	Read, Create, Edit	Read-Only	Read-Only	Read-Only (Edit with Permission Set)
<b>**Vendor Manager**</b>	Read-Only via Sharing Rule (`Event_Sharing_For_Vendors`)	No Access	Read-Only	Read, Create, Edit	No Access
<b>**Client Profile**</b>	Read-Only (Associated Events)	Read-Only (Self Profile)	No Access	No Access	No Access